

ESCS'26 Presentation Q&A Prep

Project Framing

Q: What is the main contribution of this paper?

A: The contribution is a traceable, deployment-oriented comparison of YOLO11s and RT-DETR-L for six-class PCB defect detection on the same dataset split. The paper reports aggregate metrics, per-class behavior, latency, representative successes/failures, and an online Hugging Face deployment artifact.

Q: Are you proposing a new neural network architecture?

A: No. The goal is not a new detector architecture. The goal is a reproducible applied comparison and deployment-oriented analysis for PCB inspection.

Q: Why is PCB defect detection important?

A: Small PCB defects can become expensive after assembly. A detector that can localize defects early can support quality control, repair review, and manufacturing triage.

Dataset

Q: What are the six defect classes?

A: Missing hole, Mouse bite, Open circuit, Short, Spur, and Spurious copper.

Q: How large is the dataset split?

A: The split has 5,551 training images, 1,016 validation images, and 1,016 test images. The validation split has 2,106 instances, and the test split has 2,179 instances.

Q: What image type should be used in the web demo?

A: Clean raw PCB images. Do not use screenshots, validation mosaics, or images that already contain text labels because the model may respond to those visual artifacts.

Q: Did copy-paste augmentation affect validation or test results?

A: No. Copy-paste balancing was applied to the training split only. Validation and test assignments stayed unchanged.

Models

Q: What is YOLO11s?

A: YOLO11s is a compact one-stage detector. It predicts boxes and class probabilities in one forward pass, which makes it attractive for low-latency inspection.

Q: What is RT-DETR-L?

A: RT-DETR-L is a real-time detection transformer. It uses transformer-style object queries and provides a different detector family for comparison.

Q: Why compare YOLO11s and RT-DETR-L?

A: They represent two relevant real-time detection families: YOLO-style one-stage detection and transformer-style detection. PCB defects are small and localization-heavy, so the comparison is useful for deployment choices.

Resolution Fairness

Q: Was the original comparison fair if YOLO11s used 1280 px and RT-DETR-L used 640 px?

A: The original table should be interpreted as a practical completed-configuration comparison, not a fully resolution-controlled architecture comparison. The camera-ready version addresses this by adding a matched 640-pixel comparison.

Q: What happened in the matched 640-pixel comparison?

A: YOLO11s-640 achieved 0.892 mAP50 and 0.482 mAP50-95 on the test split, while RT-DETR-L-640 achieved 0.881 mAP50 and 0.462 mAP50-95. YOLO11s remained slightly better and much faster.

Q: Did you also try RT-DETR-L at 1280 px?

A: Yes. RT-DETR-L-1280 completed at batch size 1, but it performed worse in this measured configuration: 0.604 mAP50, 0.295 mAP50-95, and 74.4 ms total latency.

Q: Why did RT-DETR-L get worse at 1280 px?

A: Higher input resolution alone does not guarantee better performance. RT-DETR-L is heavier and may require different optimization, training schedule, memory strategy, or hyperparameters at high resolution.

Q: Is the matched 640-pixel comparison fully resource-normalized?

A: No. It controls resolution, split, evaluator, hardware class, batch-1 evaluation, and no TTA. It does not fully control model capacity, FLOPs, memory, epoch count, training compute, or hyperparameter budget.

Metrics

Q: What is mAP50?

A: mAP50 is mean average precision at IoU 0.50. It is a more forgiving localization metric.

Q: What is mAP50-95?

A: mAP50-95 averages AP across IoU thresholds from 0.50 to 0.95. It is stricter and better reflects bounding-box precision.

Q: Why report both mAP50 and mAP50-95?

A: mAP50 shows whether the detector generally finds defects. mAP50-95 shows whether the boxes are tight enough for practical localization and repair review.

Q: Why use batch size 1 for evaluation?

A: Batch size 1 avoids inflated throughput from large batches and better represents an inline inspection setting where images may arrive one at a time.

Results

Q: What is the strongest practical result?

A: In the accepted practical comparison, YOLO11s-1280 achieved 0.902 mAP50, 0.502 mAP50-95, 0.866 recall, and 15.2 ms total latency on the test split.

Q: Which model is better overall?

A: Under the tested conditions, YOLO11s is the better practical choice. It has stronger recall and strict localization, and it is much faster than RT-DETR-L.

Q: Which classes are hardest?

A: Mouse bite and Spur are harder. They are small, subtle, and often narrow, so precise localization is more difficult.

Q: Why is per-class analysis important?

A: Aggregate mAP can hide weak categories. In inspection, a lower-recall class may matter a lot because missed defects can be costly.

Deployment Demo

Q: What does the Hugging Face demo show?

A: It shows a deployed YOLO11s-1280 PCB defect detector through a Gradio web interface. The demo returns annotated images, class counts, confidence scores, bounding-box coordinates, and downloadable JSON/CSV outputs.

Q: Does the demo compare YOLO11s and RT-DETR-L live?

A: No. The paper contains the controlled model comparison. The web demo shows the practical deployment of the selected YOLO11s configuration.

Q: Why is the demo latency around seconds instead of milliseconds?

A: The online demo runs on shared CPU infrastructure. Its latency should not be treated as the V100 benchmark latency reported in the paper.

Q: Is this deployed on Jetson or TensorRT?

A: No. Jetson/TensorRT benchmarking is future work only. The current deployment artifact is a Hugging Face Space.

Limitations and Future Work

Q: What are the main limitations?

A: The matched comparison is not fully resource-normalized, evaluation is in-distribution, and edge-device benchmarks were not run.

Q: What would you do next?

A: I would run real edge-device benchmarking, collect real factory images, perform resource-normalized model comparisons, and explore active high-resolution crop reinspection for small defects.

Q: What is active visual inspection?

A: It is a two-stage workflow where the system first inspects the full image, identifies uncertain or high-risk regions, then spends extra high-resolution inference only on selected crops. This may improve small-defect recall under a fixed compute budget.

Accepted Draft vs Camera-Ready (version-difference questions)

Q: The accepted draft said Spur was the best-localized class (0.631 mAP50-95), but the camera-ready says Spur is the hardest (0.381). Which is right?

A: The camera-ready is authoritative. During camera-ready reproducibility work we found that an earlier dataset export changed the displayed class-name order without remapping label IDs, so per-class rows for Missing_hole, Mouse_bite, and Spur could be misnamed in the accepted draft. Aggregate metrics are class-agnostic and were unaffected. The camera-ready per-class table is generated only from the corrected canonical mapping (and from the matched 640-pixel models), which is why the per-class picture changed. This correction is disclosed in the response to reviewers. (Backup slide B2.)

Q: The accepted draft reported RT-DETR-L total latency as 31.9 ms, but the camera-ready says 49.8 ms. Why?

A: The draft number came from an earlier measurement pass. For the camera-ready, all models were re-measured under the pinned protocol — batch size 1, zero dataloader workers, no TTA, Ultralytics 8.4.51, PyTorch 2.5.1, single V100 — and those values supersede the draft. The conclusion never changed: YOLO11s was roughly 3x faster in every measurement pass. (Backup slide B7.)

Q: What happened to the WBF fusion and inspection-cost results from the accepted paper?

A: They are preserved as supporting context from the accepted study's bundle but were not re-run inside the unified batch-1 evaluator, so the camera-ready keeps its headline on single-model comparisons where resolution and architecture effects stay interpretable. The numbers are still available: adaptive WBF lifts recall to 0.903, balanced WBF to 0.943 at 0.85 FP/image, and YOLO11s alone is cost-optimal when misses and false alarms are weighted equally. (Backup slide B6.)

Q: Instead of retraining at 640, couldn't you just evaluate the 1280 checkpoint at 640?

A: We ran exactly that diagnostic. Scoring the YOLO11s-1280 checkpoint at 640 without retraining drops recall from 0.866 to 0.810 and mAP50-95 from 0.502 to 0.453. That is why the matched comparison uses a YOLO11s model actually trained at 640; resize-only evaluation is not a substitute for resolution-specific training. (Backup slide B3.)

Q: Do the validation-split numbers agree with the test-split story?

A: Yes. On validation, YOLO11s-1280 scores 0.874 recall / 0.505 mAP50-95 vs RT-DETR-L-640 at 0.855 / 0.472, with RT-DETR-L fractionally ahead on precision — the same pattern as the test split, so the headline is not a split artifact. (Backup slide B4.)

Day-of Checklist

- Record the presentation for Prof. Inventado: start Zoom local/cloud recording before the talk begins (or run a QuickTime screen recording as backup), and verify audio is captured.
- Have open and ready to screen-share: the main deck, `escs26_qa_backup_slides.pptx`, the camera-ready PDF (`reports/publication/escs26_camera_ready.pdf`), and the offline demo video (in case the Hugging Face Space is slow on shared CPU).
- Prof. Inventado will try to join the 5:20 PM session; send him the recording afterward either way.

Short Answers For Stressful Moments

Q: What is your one-sentence takeaway?

A: YOLO11s is the stronger deployment-oriented detector in our tested PCB inspection setting, and the camera-ready experiments show that this conclusion still holds under a matched 640-pixel comparison.

Q: What is the biggest caveat?

A: The study is not a full embedded-device benchmark; V100 results and Hugging Face CPU demo latency should not be confused with Jetson/TensorRT performance.

Q: Why should someone trust the results?

A: The same split, evaluator, batch-1 protocol, no TTA, saved CSVs, per-class metrics, and reproduced baseline checks were used to make the comparison traceable.

Q: What should I say if I do not know an answer?

A: “That is a good point. I did not test that condition in this study, so I would treat it as future work rather than speculate beyond the measured results.”